# Reporte Técnico de Actividades Práctico-Experimentales Nro. 3

## 1. Datos de Identificación del Estudiante y la Práctica

| | |
|---|---|
| **Nombre del estudiante(s)** | - Anthony Yaguana<br>- Derick Vargas<br>- Darwin Correa<br>- Bryan Troya |
| **Asignatura** | Sistemas Digitales |
| **Ciclo** | 5 |
| **Unidad** | 1 – Microcontroladores para sistemas embebidos |
| **Resultado de aprendizaje de la unidad** | R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 3 |
| **Título de la Práctica** | Temporizador de hardware y generación de señal con ESP32 |
| **Nombre del Docente** | Luis Darío Sinche C. |
| **Fecha** | 30 de octubre de 2026 |
| **Horario** | Viernes 11h30 a 13h30 |
| **Lugar** | Laboratorio de computación<br>(simulación en Wokwi y prueba en la placa ESP32 del grupo) |
| **Tiempo planificado en el Sílabo** | 3 horas (APE, semana 3): 2 h presenciales y 1 h para terminar el informe |
| **Placa** | ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32) |

## 2. Objetivo(s) de la Práctica

**Objetivo:** Configurar un temporizador de hardware del ESP32 con la API del core 3.x, usar su interrupción para hacer parpadear un LED y generar una onda cuadrada de frecuencia calculada, y medir esa señal con un analizador lógico.

Al terminar, la pareja podrá:

- Calcular el valor de alarma para un tiempo dado.
- Explicar el recorrido completo: el contador llega a la alarma, se produce la interrupción, se ejecuta la ISR y loop() responde.
- Medir la frecuencia de una señal digital y compararla con la calculada.

## 3. Materiales, Reactivos, Equipos y Herramientas

- Archivo: APE 3 Guía
- Hardware (por grupo, solo para la Parte C):

| Cantidad | Componente |
|---|---|
| 1 | ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga) |
| 1 | Protoboard |
| 1 | LED rojo de 5 mm |
| 1 | Resistencia de 330 Ω |
| Varios | Cables jumper macho-macho |

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE con el paquete esp32 by Espressif Systems versión 3.x, sin librerías externas, para la Parte C.
- PulseView (gratuito, sigrok.org) o GTKWave (gratuito), para abrir el archivo
- vcd. Instálenlo antes de la práctica.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

## 4. Procedimiento / Metodología Ejecutada

- **Parte A**
  - Se armó el circuito en Wokwi conforme al Anexo A ESP32, LED, resistencia y analizador ya cableados.
  - Se predijo el tiempo entre cambios: con ALARMA_US = 1000000` el LED cambiaría cada 1000 ms periodo completo 2000 ms.
  - Se inició la simulación y se observó el LED.
  - Se restaron dos instantes consecutivos del Monitor y se comprobó que coincidía con lo calculado.
  - Se detuvo la simulación y se cambió ALARMA_US a 250000 y se repitió la prueba cambio cada 250 ms.
  - Se repitió nuevamente con ALARMA_US = 50000 y cambio cada 50 ms.
  - Se tomaron capturas del Monitor Serie para cada valor.

- **Parte B**
  - Se calcularon los tiempos de alarma teóricos para las frecuencias de 100 Hz, 1000 Hz y 5000 Hz.
  - Se configuró el código con cada frecuencia y se ejecutó la simulación en Wokwi.
  - Se registraron los cambios de estado por segundo mediante el Monitor Serie para determinar la frecuencia medida.
  - Se exportó y analizó el archivo VCD del analizador lógico para medir el periodo exacto de la señal.
  - Se compararon los resultados experimentales con los teóricos y se completó la Tabla B.
- **Parte C**
  - Se armó el la placa conforme a la simulación en Woki
  - Se cargó el código en Arduino IDE

- Se seleccionó la placa y cargó el código en la placa
- Se comprobó el correcto funcionamiento de la placa
- Se midió la frecuencia con el multímetro prestado en el laboratorio

- **Parte D**
  - Se documenta toda la práctica en el repositorio indicado en la sección de Anexos.

## 5. Resultados

- **Parte A**

  **¿cada cuántos milisegundos cambiará el LED? Anoten la predicción en la Tabla A.**

  **Fórmulas**

  tCambio $=ALARMA_US \times 1\mu s$

  Tparpadeo$=2 \times ALARMA_US$

  **Predicciónes**

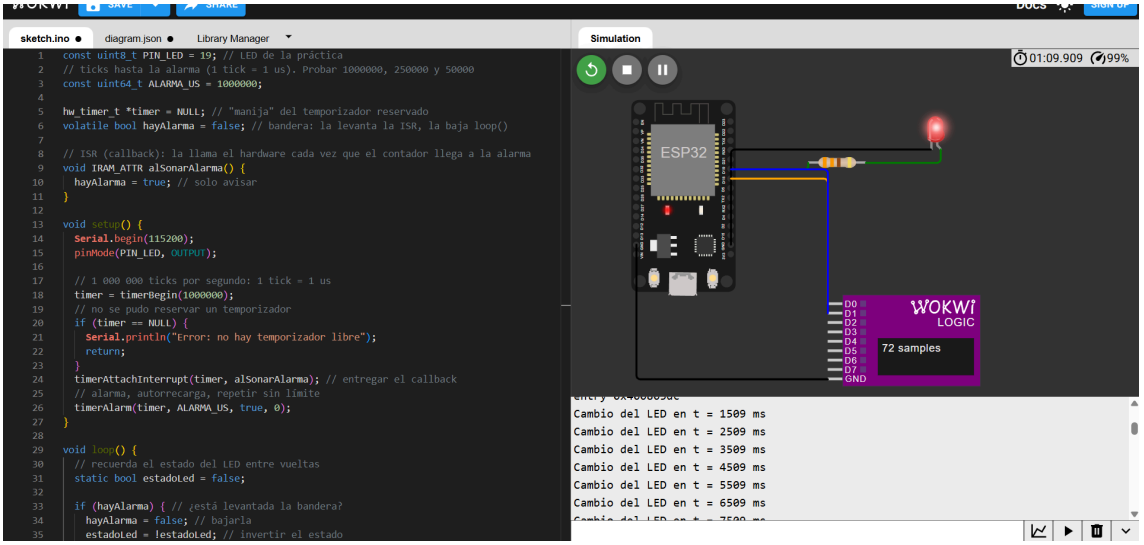
$$t = \frac{1000000}{1000} = 1000ms$$

$$t = \frac{250000}{1000} = 250ms$$

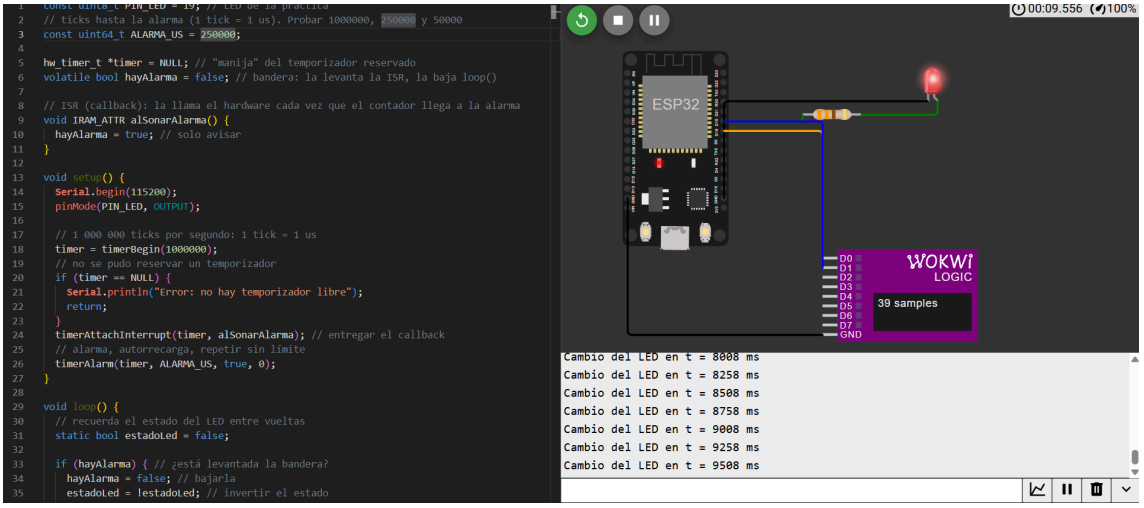
$$t = \frac{50000}{1000} = 50ms$$

  Predicción
  LED cambia cada 1 s
  LED cambia cada 0,25 s
  LED cambia cada 0,05

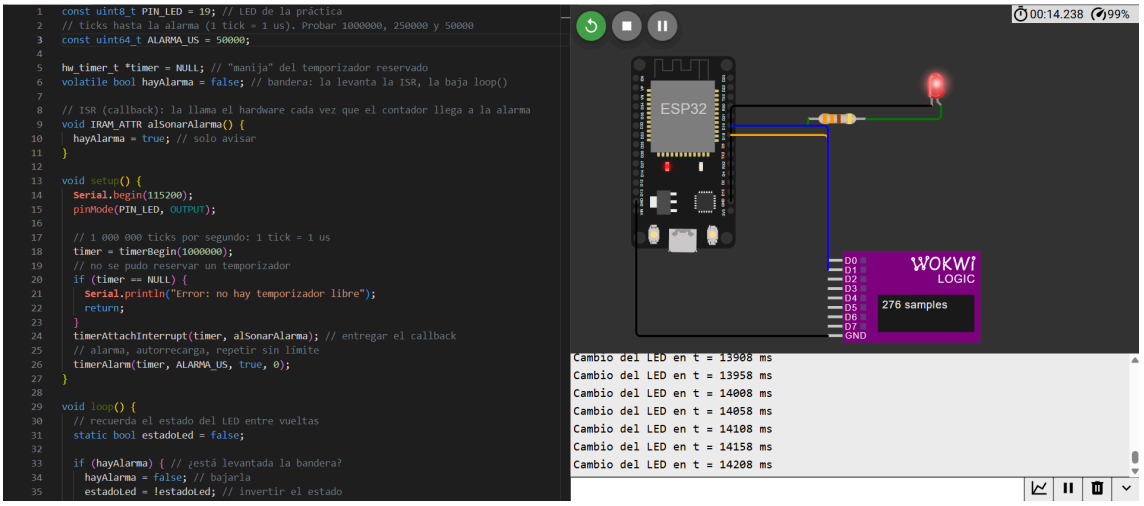
## 10000000




## 250000



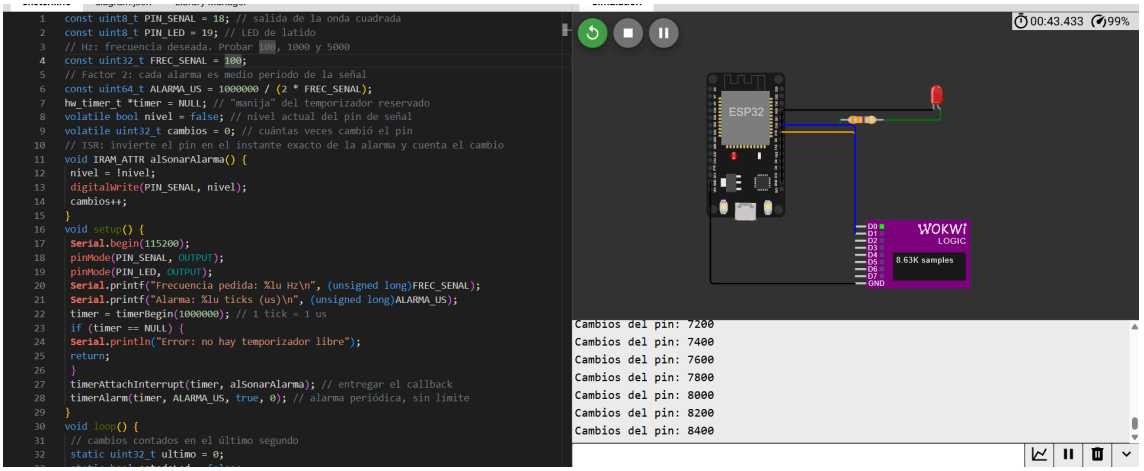

## 50000



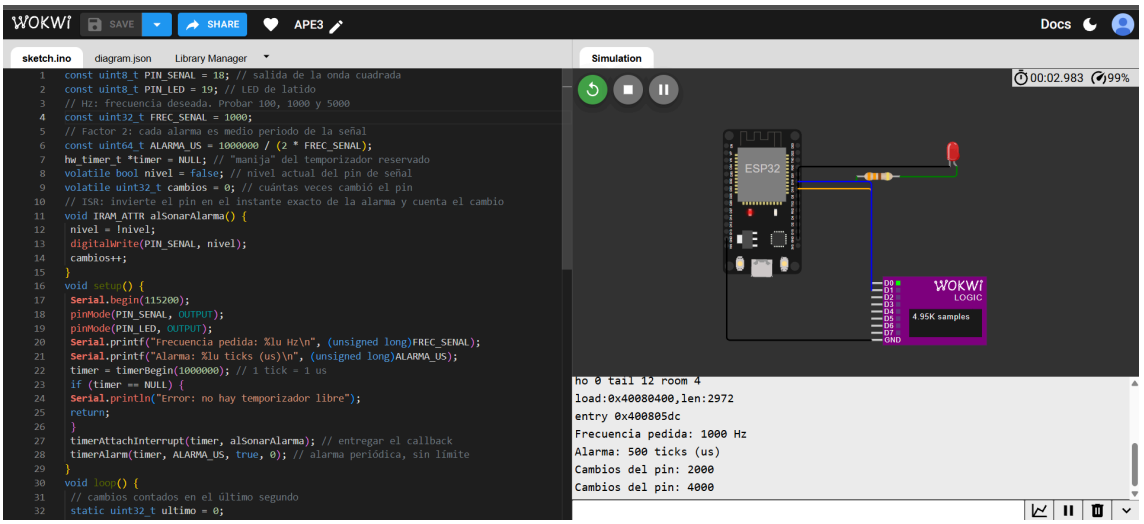
| ALARMA_US (ticks) | Tiempo calculado entre cambios (ms) | Tiempo medido en el Monitor Serie (ms) | Periodo completo del parpadeo (ms) |
|---|---|---|---|
| 1 000 000 | 1000ms | 1000ms | 2000ms |
| 250 000 | 250ms | 250ms | 500ms |
| 50 000 | 50ms | 50ms | 100ms |

- **Parte B**

**FREC_SENAL = 100**




**FREC_SENAL = 1000**



**FREC_SENAL = 5000**

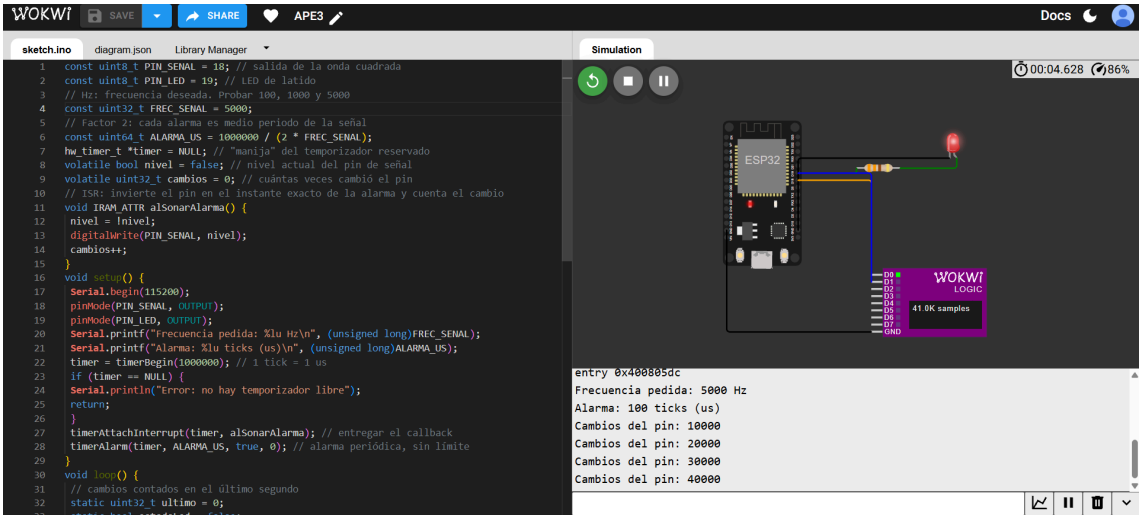
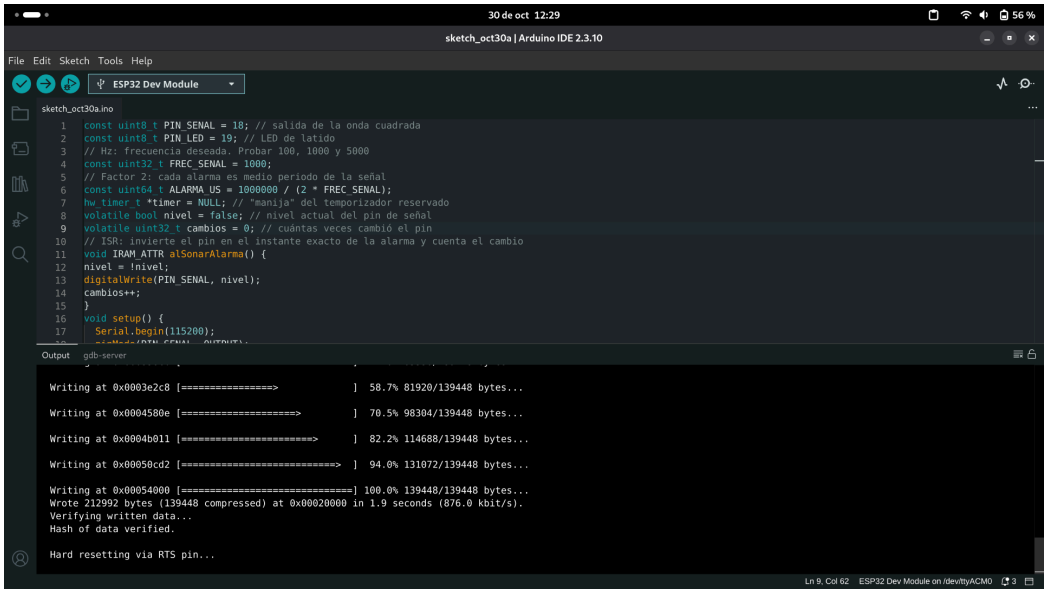
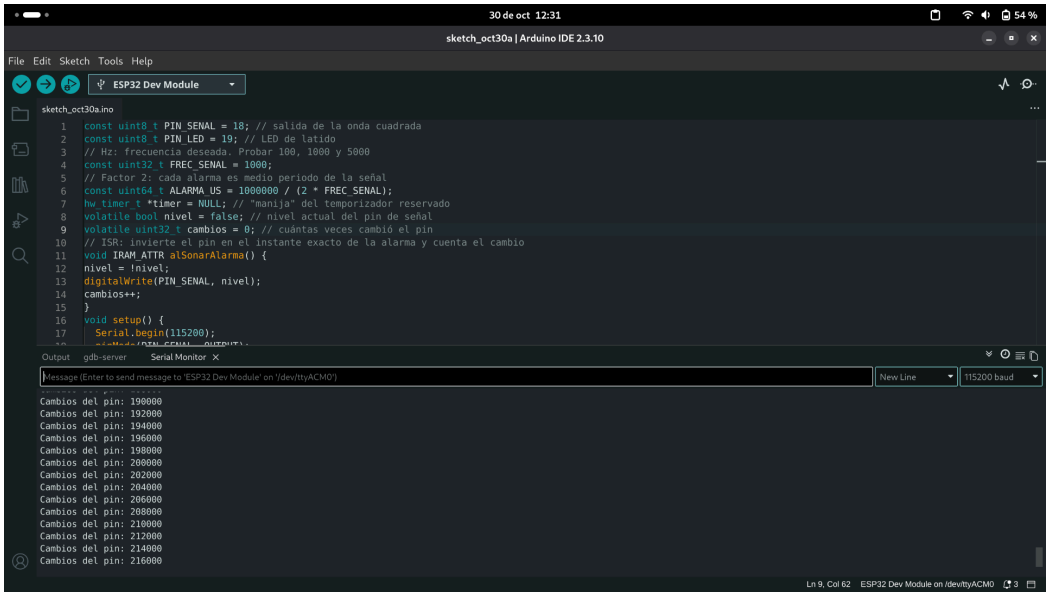


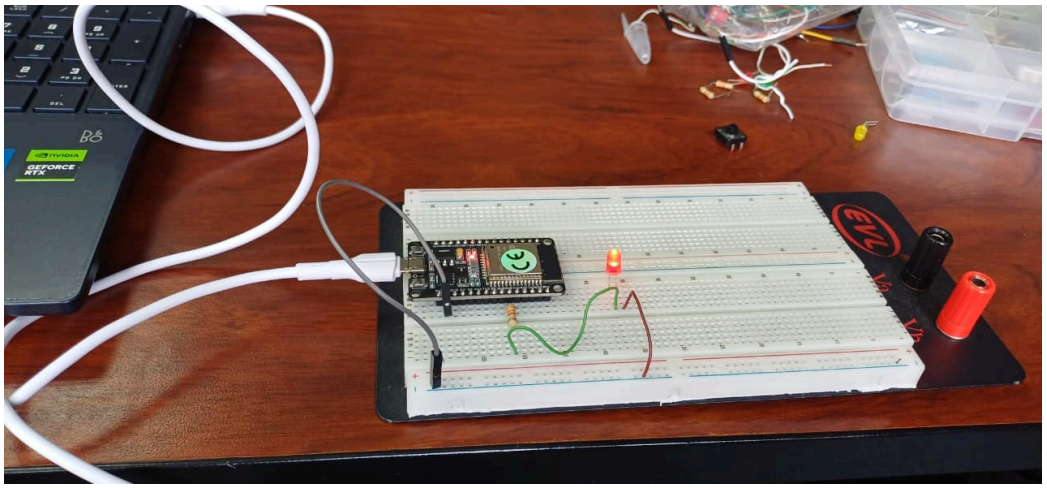

**Tabla B. Generación y medición de la onda cuadrada**

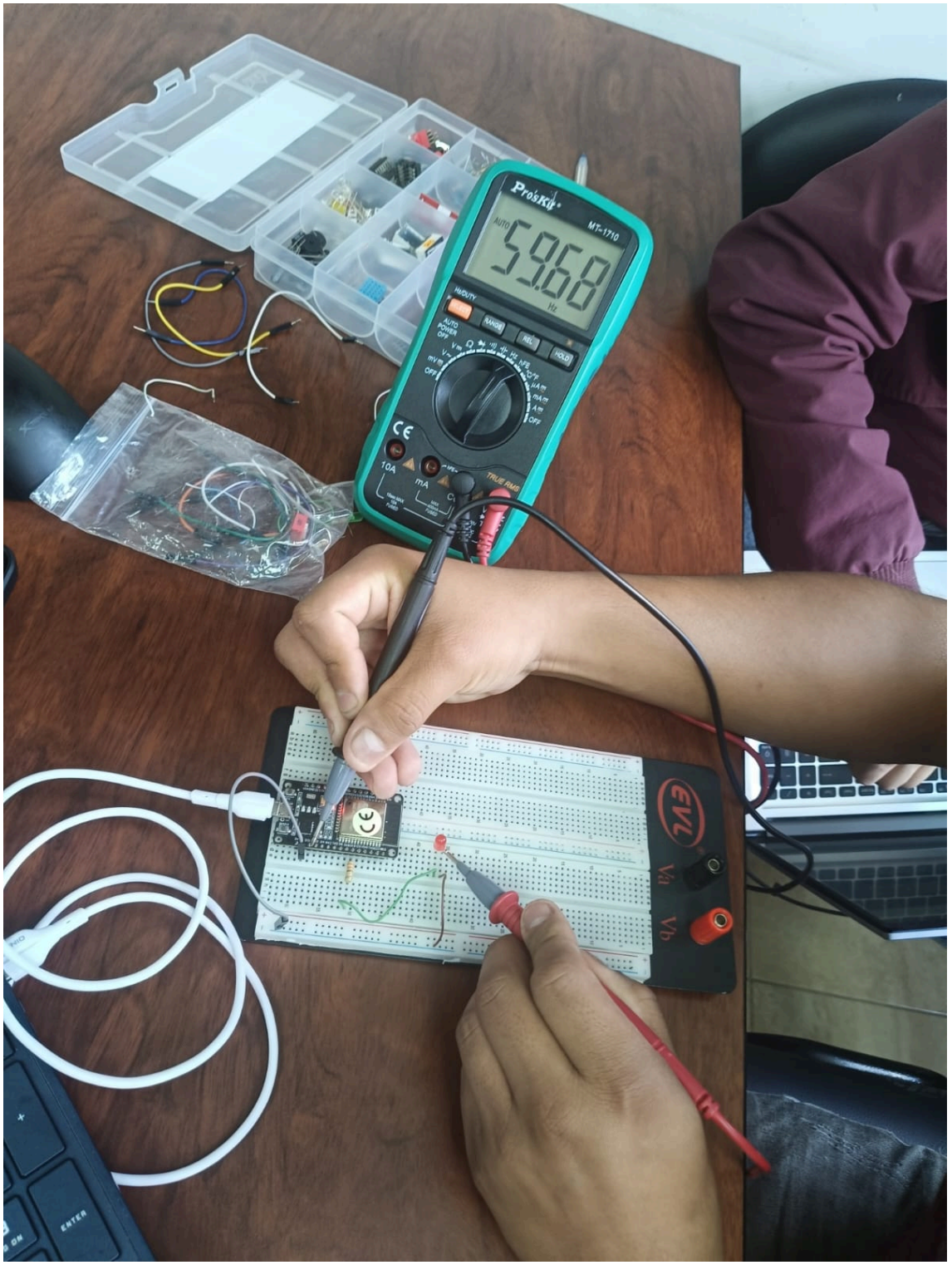
| FREC_SENAL pedida (Hz) | ALARMA_US calculada a mano | ALARMA_US impresa por el programa | Cambios por segundo (Monitor Serie) | Frecuencia medida = cambios / 2 (Hz) | Periodo medido en PulseView (ms) |
|---|---|---|---|---|---|
| 100 | 5000 us | 5000 us | 200 | 100 Hz | - |
| 1000 | 500 us | 500 us | 2000 | 1000 Hz | 1 ms |
| 5000 | 100 us | 100 us | 10000 | 5000 Hz | 0.201 ms |

- **Parte C**




30 de oct 12:31
54 %
sketch_oct30a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
ESP32 Dev Module
sketch_oct30a.ino
const uint8_t PIN_SENAL = 18; // salida de la onda cuadrada
const uint8_t PIN_LED = 19; // LED de latido
// Hz: frecuencia deseada. Probar 100, 1000 y 5000
const uint32_t FREC_SENAL = 1000;
// Factor 2: cada alarma es medio periodo de la señal
const uint64_t ALARMA_US = 1000000 / (2 * FREC_SENAL);
hw_timer_t *timer = NULL; // "manija" del temporizador reservado
volatile bool nivel = false; // nivel actual del pin de señal
volatile uint32_t cambios = 0; // cuántas veces cambió el pin
// ISR: invierte el pin en el instante exacto de la alarma y cuenta el cambio
void IRAM_ATTR alSonarAlarma() {
nivel = !nivel;
digitalWrite(PIN_SENAL, nivel);
cambios++;
}
void setup() {
Serial.begin(115200);
Output gdb-server Serial Monitor
Message (Enter to send message to 'ESP32 Dev Module' on '/dev/ttyACM0')
New Line
115200 baud
Cambios del pin: 190000
Cambios del pin: 192000
Cambios del pin: 194000
Cambios del pin: 196000
Cambios del pin: 198000
Cambios del pin: 200000
Cambios del pin: 202000
Cambios del pin: 204000
Cambios del pin: 206000
Cambios del pin: 208000
Cambios del pin: 210000
Cambios del pin: 212000
Cambios del pin: 214000
Cambios del pin: 216000
Ln 9, Col 62 ESP32 Dev Module on /dev/ttyACM0






- **Parte D**
  Repositorio:
  https://github.com/Anthony-YP/APE2-SistemasDigitales-Yaguana-Troya-Correa-vargas.git

## 6. Preguntas de Control

Responda a las preguntas del docente.

**1. En el Código 1, la ISR solo pone hayAlarma = true. ¿Por qué no se imprime el mensaje directamente dentro de la ISR?**

Porque la ISR debe ejecutarse lo más rápido posible. Imprimir mensajes con Serial.print() puede tardar y afectar el funcionamiento de las interrupciones. Por eso, la ISR únicamente activa una bandera y el loop() se encarga de mostrar el mensaje en el Monitor Serie.

**2. Con timerBegin(1000000) y ALARMA_US = 250000, ¿cada cuánto cambia el LED? ¿Cuánto dura el periodo completo del parpadeo? Comparen con su Tabla A.**

El temporizador trabaja con 1 000 000 ticks por segundo, por lo que cada tick dura 1 µs.

Tiempo entre cambios:

250 000 / 1000 = **250 ms**

Periodo completo:

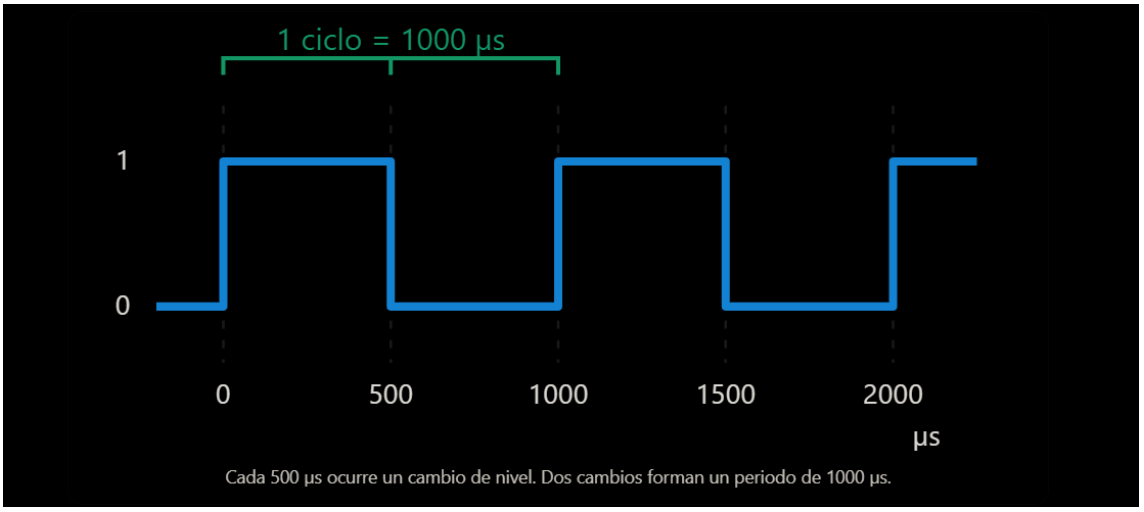
T = 250 × 2 = **500 ms**

Al comparar con la Tabla A, el tiempo medido fue de 250 ms, coincidiendo con el valor calculado.

**3. Si se quita la palabra volatile de la variable hayAlarma, ¿qué podría pasar y por qué?**

El compilador podría optimizar el acceso a la variable y no detectar correctamente los cambios realizados por la interrupción. Esto podría provocar que el loop() no responda a las alarmas como se espera y que el LED no cambie de estado correctamente.

**4. Expliquen con un dibujo por qué, con una alarma cada 500 µs, la señal del GPIO18 tiene 1 kHz y no 2 kHz.**



Cada alarma cambia el estado de la señal, pero se necesitan dos cambios para completar un ciclo: uno para pasar a nivel alto y otro para regresar al nivel bajo.

Tiempo de un ciclo:

T = 500 µs + 500 µs = 1000 µs = 0,001 s

Frecuencia:

f = 1 / 0,001 = **1000 Hz = 1 kHz**

Por lo tanto, la frecuencia es de 1 kHz y no de 2 kHz.

**5. Calculen ALARMA_US para una señal de 2 kHz y para una de 250 Hz.**

Fórmula:

ALARMA_US = 1 000 000 / (2 × f)

Para 2 kHz (2000 Hz):

ALARMA_US = 1 000 000 / (2 × 2000) = **250 ticks**

Para 250 Hz:

ALARMA_US = 1 000 000 / (2 × 250) = **2000 ticks**

**6. ¿Coinciden las frecuencias medidas en la Tabla B con las pedidas? Si hay diferencias, ¿a qué las atribuyen?**

*Pendiente de completar con las mediciones de la Parte B.* Se deben comparar las frecuencias solicitadas de 100 Hz, 1000 Hz y 5000 Hz con los valores obtenidos del Monitor Serie. Si existen diferencias, se deben analizar posibles variaciones de temporización o del proceso de medición.

**7. Con timerBegin(1000000), ¿qué valor de prescaler N calcula el core? Muestren el cálculo a partir del reloj de 80 MHz.**

El reloj del temporizador tiene una frecuencia de 80 MHz, equivalente a 80 000 000 Hz.

Se desea trabajar con una frecuencia de 1 000 000 Hz.

N = 80 000 000 / 1 000 000

**N = 80**

Por lo tanto, el prescaler es 80, lo que permite que el temporizador cuente un tick cada microsegundo

**8. En esta práctica, ¿por qué el contador del temporizador nunca llega a desbordarse?**

Porque el temporizador está configurado con autorrecarga mediante timerAlarm(timer, ALARMA_US, true, 0). Cuando llega al valor de alarma, vuelve a iniciar su conteo desde cero. De esta manera, el contador se reinicia periódicamente antes de alcanzar su límite máximo.

## 7. Conclusiones

- Mediante las simulaciones realizadas en Wokwi, se comprobó que el temporizador de hardware del ESP32 permite controlar los intervalos de cambio de estado de un LED. Para las alarmas de 1 000 000, 250 000 y 50 000 ticks, se obtuvieron tiempos medidos de 1000 ms, 250 ms y 50 ms, respectivamente, los cuales coincidieron con los valores calculados. Además, se determinó que los periodos completos del parpadeo fueron de 2000 ms, 500 ms y 100 ms.
- Como la ISR invierte el GPIO18 en cada alarma, cada alarma produce medio periodo; por eso la frecuencia de la señal es la mitad de la frecuencia de alarmas y se aplica ALARMA_US = 1 000 000 / (2 × FREC_SENAL). Con esta fórmula se obtuvieron contadores de 5000, 500 y 100 ticks para 100, 1000 y 5000 Hz, y las frecuencias medidas con el Monitor Serie (cambios/2) coincidieron con las pedidas (Tabla B). A 1 kHz, el analizador lógico de Wokwi y PulseView mostraron un periodo de ≈1 ms, coherente con T = 1/f.
- Tanto en Wokwi como en la placa ESP32 (Parte C, FREC_SENAL = 1000) el LED de latido parpadeó una vez por segundo y el GPIO18 generó una señal cercana a 1 kHz. Las pequeñas desviaciones observadas se explican por el redondeo entero de ALARMA_US, la tolerancia del cristal del ESP32, la resolución de millis() (1 ms) y la precisión del instrumento de medida; ninguna altera el comportamiento esperado del sistema.

## 8. Bibliografía / Referencias

- Diapositivas semana 3

## 9. Anexos

- Repositorio: https://github.com/Anthony-YP/APE2-SistemasDigitales-Yaguana-Troya-Correa-vargas.git
- Link de parte A:https://wokwi.com/projects/479327914427670529
- Link de parte B: https://wokwi.com/projects/479324852811415553

## 10. Declaración de uso de la IA

Durante el desarrollo de la práctica se utilizó la herramienta de inteligencia artificial ChatGPT como apoyo para mejorar la redacción de algunos apartados del informe, verificar los cálculos y resultados obtenidos en las simulaciones realizadas en Wokwi y aclarar dudas relacionadas con el funcionamiento de los temporizadores, las interrupciones y la generación de ondas cuadradas en el ESP32.

Asimismo, se utilizó como herramienta de consulta para comprender algunas preguntas de control y resolver inquietudes que surgieron durante el trabajo grupal. Las simulaciones, la recopilación de evidencias y la revisión de los resultados fueron realizadas por los integrantes del grupo, utilizando la inteligencia artificial únicamente como recurso de apoyo académico.